

Permisos – Formato octal y formato simbólico

Cada archivo y cada directorio dentro del sistema operativo tienen asignado un grupo de permisos con el fin de establecer quién puede o no acceder a esa información. Algunos permisos establecerán que usuario o grupos de usuario pueden Leer, escribir o ejecutar ese archivo, o bien solo alguno de ellos.

Existen 2 formas de asignar permisos a los archivos o directorios, en formato octal y en formato simbólico. Asimismo, existen 3 tipos de permisos especiales que refuerzan estos permisos: SetUID, SetGID y StickyBit.

Formato octal

El formato octal es una forma numérica de representar los permisos de archivos y directorios en Linux. Cada conjunto de permisos (lectura, escritura y ejecución) para el usuario, el grupo y otros se traduce a un valor número que permite una forma más concisa y precisa de manipularlos con comandos en la terminal.

Los permisos se dividen en 3 grupos para cada archivo o directorio:

- 1) **Usuario (*Owner*)**: el propietario del archivo.
- 2) **Grupo (*Group*)**: los miembros del grupo asociado al archivo.
- 3) **Otros (*Others*)**: todos los demás usuarios del sistema.

Los permisos se representan como una cadena de caracteres de 10 posiciones.

Ejemplo

```
-rwxr-xr--
```

El primer carácter indica el tipo de archivo “-” para archivos regulares, “d” para directorios.

Los posibles valores para esta posición son los siguientes:

Comando	Descripción
-	«archivo regular» - un guión representa un archivo común (de texto, html, .mp3, .jpg, etc.)
d	«directorio» representa un directorio.
l	«link», es decir un enlace o acceso directo.
b	«archivos de bloque», archivos que representan hardware. Usualmente en /dev.
c	«archivos de caracter», proveen un stream serial para entrada o salida. Las terminales son ejemplo. Se hallan en /dev.
p	«archivo de pipe», archivo FIFO o «named pipes», o tuberías, no ocupan lugar en el espacio y son como un punto de referencia en el filesystem al solo efecto de que dos procesos intercambien información. Se crean con <code>CMD mkfifo</code> . ³

CUADRO 5.8: TIPOS DE ARCHIVO

Fuente: Niklas, P. (2020). *GNU/Linux, Con sabor a Debian*. Pág. 108.

Las siguientes posiciones se agrupan de tres en tres:

- 1) rwx: permisos de **Lectura** (*read*), **Escritura** (*write*) y **Ejecución** (*execute*) para el usuario propietario (*owner*) del archivo.
- 2) r-x: permisos de **Lectura** (*read*) y **Ejecución** (*execute*) para el grupo (*group*) asociado al archivo.
- 3) r--: permisos de **Lectura** (*read*) para otros (*other*).

Nota: recordamos que la forma de listar archivos y que se vean los permisos es con el comando “ls -l”.

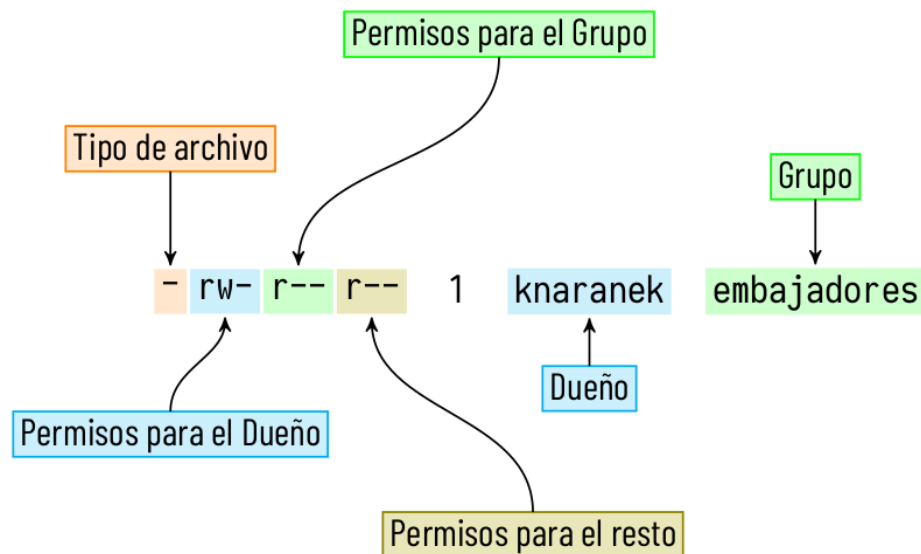


FIGURA 5.2: DISTRIBUCIÓN DEL ESQUEMA DE PERMISOS

Fuente: Niklas, P. (2020). *GNU/Linux, Con sabor a Debian*. Pág. 109.

A cada uno de estos permisos se le asigna un valor numérico basado en su peso binario:

- $r = 4$
- $w = 2$
- $x = 1$

Esto quiere decir que la asignación de permisos en número octal es la suma de los valores de los permisos que posee.

Ejemplo

- $rwX = 7 (4 + 2 + 1)$
- $rw- = 6 (4 + 2)$
- $r-x = 5 (4 + 1)$
- $r-- = 4$

Salvo por el primer espacio, de las nueve posiciones de permisos que administramos, son en realidad un bit que está encendido o apagado, teniendo en cuenta que si este está encendido, mostrara una letra que lo representa, de lo contrario un guión. La combinación

de bits encendidos o apagados en cada grupo da ocho posibles combinaciones de valores ($2^3 = 8$), es decir, la suma de los bits encendidos:

Representación	Binario	Valor octal	Descripción
---	000	0	Sin permisos
--x	001	1	Ejecución
-w-	010	2	Escritura
-wx	011	3	Escritura y ejecución
r--	100	4	Lectura
r-x	101	5	Lectura y ejecución
rw-	110	6	Lectura y escritura
rwX	111	7	Lectura, escritura y ejecución

Comandos para la asignación de permisos en formato octal

El comando que utilizaremos para la asignación de los permisos es “chmod”:

```
chmod 754 archivo
```

Formato simbólico

Otra forma de asignar permisos es por medio del formato simbólico a través de una cadena de caracteres. Esta representación es muy visual y ayuda a darle una comprensión más sencilla a la asignación de los permisos.

El comando para establecer los permisos en formato simbólico sigue siendo “chmod”, pero se agrega a **quien** se le asigna el permiso y los operadores “+” ; “-”; “=” (más, menos e igual) para añadir o quitar el permiso representado en valor **r**, **w** o **x**.

```
chmod “quién” “operador” “permiso” archivo
```

quien:

- **u:** *owner*
- **g:** grupo
- **o:** otros

operador:

- **+**: añadir
- **-**: quitar
- **=**: establecer permiso específico

Ejemplo

```
chmod u+rw archivo
```

```
chmod g-w archivo
```

```
chmod a=rx archivo
```

Cambio de propietario y grupo

Como vimos en los ejemplos anteriores, cada archivo o directorio, tiene un *owner* y grupo con derechos. Estos dos componentes (usuario y grupo) pueden ser reemplazados por otros de acuerdo a la necesidad utilizando los comandos “chown” y “chgrp”.

```
-rw-r--r-- 1 root root 65 Oct 6 11:34 salida.txt
```

```
-rw-r--r-- 1 root root 45 Oct 6 14:52 frutas.txt
```

Ejemplo

```
chown “usuarios” archivo
```

```
chgrp “grupo” archivo
```

```
-rw-r--r-- 1 alumno alumno 65 Oct 6 11:34 salida.txt
```

```
-rw-r--r-- 1 alumno Tecnologia 45 Oct 6 14:52 frutas.txt
```

El usuario *root* podría cambiar ambos en una sola sentencia pasando usuario y grupo separados por dos puntos, de la siguiente forma:

```
chown alumno: alumno salida.txt
```

Bibliografía

- Niklas, P. (2020). *GNU/Linux, con sabor a Debian*. Pp. 107 (5.7) - 119 (5.6).